

Курсовая работа

Задача 1.4 — экспорт `dm.dm_f101_round_f` в CSV и импорт обратно в `dm.dm_f101_round_f_v2`

Студент: Орисенко Максим

Дата: 09.05.2026

1. Постановка задачи

После того как в задаче 1.3 была рассчитана 101-я форма (`dm.dm_f101_round_f`) — менеджер проекта доволен, ЦБ получил отчёт. Теперь нужно подготовить инфраструктуру обмена этими данными между отделами: написать два скрипта — экспорт витрины в CSV и импорт CSV обратно в БД (в копию таблицы `dm.dm_f101_round_f_v2`).

Конкретно:

- написать скрипт, который выгружает данные из `dm.dm_f101_round_f` в CSV-файл; первая строка — имена колонок таблицы;
- убедиться, что данные выгружаются в корректном формате;
- написать скрипт, который импортирует CSV обратно в БД, в копию таблицы `dm.dm_f101_round_f_v2`;
- изменить пару значений в выгруженном CSV-файле и загрузить его в `dm.dm_f101_round_f_v2` — это и продемонстрирует, что импорт работает корректно (изменения в файле должны попасть в `_v2`);
- покрыть оба процесса простым логированием.

2. Стек и архитектура решения

Стек тот же, что и в предыдущих задачах: PostgreSQL 13 в Docker (контейнер `hw_coursework_postgres`, порт 5433 проброшен на хост, БД `airflow`, пользователь `airflow`, пароль `airflow`), Python 3 + `psycopg2`, DBeaver — для проверки результатов и логов. Скрипты языком выбран Python, потому что весь предыдущий ETL курсовой написан на нём же — так все артефакты задачи остаются однородными.

Архитектурное решение по импорту: для загрузки CSV используется серверный `COPY ... FROM STDIN` (`psycopg2.cursor.copy_expert`) — это самый быстрый путь массовой загрузки в Postgres. Предварительно делаем `TRUNCATE` целевой таблицы, чтобы импорт был идемпотентным и его можно было запускать многократно.

```
PS C:\Users\Maksim\Desktop\neo\hw_coursework_task1\python> docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

NAMES	STATUS	PORTS
hw_coursework_postgres	Up 56 minutes	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp

Рис. 1. Контейнер `hw_coursework_postgres` запущен, проброшен порт 5433 → 5432.

В DBeaver видны три рабочие схемы — `ds`, `logs` и `dm`; в `dm` уже лежит 101-я форма из задачи 1.3 (`dm_f101_round_f`) и страховочная копия (`dm_f101_round_f_v2`), куда мы будем импортировать CSV:

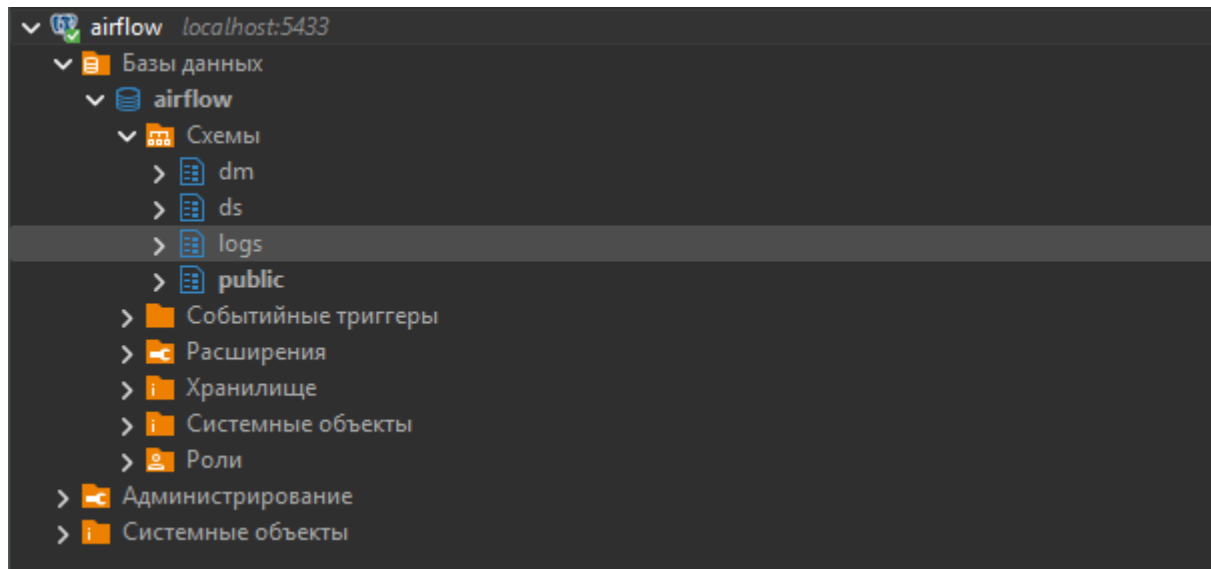


Рис. 2. Дерево схем БД airflow в DBeaver: dm, ds, logs, public.

3. Структура проекта (фокус на 1.4)

hw_coursework_task1/

├─ python/

| └─ db.py параметры подключения

| └─ export_f101.py выгрузка dm.dm_f101_round_f → CSV

| └─ import_f101.py загрузка CSV → dm.dm_f101_round_f_v2

└─ out/ артефакты выгрузки (в .gitignore)

└─ dm_f101_round_f.csv свежая выгрузка (создаётся скриптом)

└─ dm_f101_round_f_modified.csv копия с подменёнными значениями для демо импорта

4. Скрипт экспорта (export_f101.py)

Логика скрипта:

- выставляем целевой файл `OUT_FILE = ../out/dm_f101_round_f.csv` (путь нормализуется через `os.path.normpath`, чтобы в логах были аккуратные абсолютные пути);
- логируем событие `START` в `logs.etl_log` с полем `extra` = путь к выходному файлу;
- делаем `SELECT * FROM dm.dm_f101_round_f ORDER BY ledger_account` и читаем имена колонок из `cur.description`;
- записываем в CSV: первая строка — имена колонок, далее построчно сами данные; разделитель = `;` (чтобы запятые в десятичных значениях, если бы они вдруг появились, не мешали парсингу), кодировка `utf-8`;
- `Decimal`-значения форматируются вспомогательной функцией `fmt()` через `format(value, "f")` — это превращает «`0E-8`» (научная нотация `Decimal`) в обычное «`0.00000000`»; читать такой CSV в Excel/

редакторе намного удобнее;

- логируем событие END с `rows_affected = len(rows)`;
- печатаем в `stdout` краткое подтверждение: путь и число строк.

```
1  from __future__ import annotations
2
3  import csv
4  import os
5  from datetime import datetime
6  from decimal import Decimal
7
8  import psycopg2
9
10 from db import DB_CONFIG
11
12 OUT_DIR = os.path.normpath(os.path.join(os.path.dirname(__file__), "..", "out"))
13 OUT_FILE = os.path.join(OUT_DIR, "dm_f101_round_f.csv")
14
15
16 def fmt(value):
17     """Привести Decimal к обычной строке с фиксированной точкой."""
18     if isinstance(value, Decimal):
19         return format(value, "f")
20     return value
21
22
23 def main() -> None:
24     os.makedirs(OUT_DIR, exist_ok=True)
25     started = datetime.now()
26
27     with psycopg2.connect(**DB_CONFIG) as conn, conn.cursor() as cur:
28         cur.execute(
29             "INSERT INTO logs.etl_log(process,object_name,event,started_at,extra) "
30             "VALUES (%s,%s,%s,%s,%s)",
31             ("export_f101", "dm.dm_f101_round_f", "START", started, OUT_FILE),
32         )
33     cur.execute("SELECT * FROM dm.dm_f101_round_f ORDER BY ledger_account;")
34     cols = [c.name for c in cur.description]
35     rows = cur.fetchall()
36
37     with open(OUT_FILE, "w", encoding="utf-8", newline="") as f:
38         w = csv.writer(f, delimiter=";")
39         w.writerow(cols)
40         for r in rows:
41             w.writerow([fmt(v) for v in r])
42
43     cur.execute(
44         "INSERT INTO logs.etl_log(process,object_name,event,started_at,finished_at,rows_affected,extra) "
45         "VALUES (%s,%s,%s,%s,%s,%s,%s)",
46         ("export_f101", "dm.dm_f101_round_f", "END",
47          started, datetime.now(), len(rows), OUT_FILE),
48     )
```

Рис. 3. `export_f101.py` — общий вид: `imports`, нормализация пути, `fmt()` для `Decimal`, основной блок с подключением, `SELECT`, `csv.writer` и логированием `START/END`.

Крупным планом — верхняя часть скрипта (`imports`, нормализация пути, функция `fmt`):

```

1  from future import annotations
2
3  import csv
4  import os
5  from datetime import datetime
6  from decimal import Decimal
7
8  import psycopg2
9
10 from db import DB_CONFIG
11
12 OUT_DIR = os.path.normpath(os.path.join(os.path.dirname(__file__), "..", "out"))
13 OUT_FILE = os.path.join(OUT_DIR, "dm_f101_round_f.csv")
14
15
16 def fmt(value):
17     """Привести Decimal к обычной строке с фиксированной точкой."""
18     if isinstance(value, Decimal):
19         return format(value, "f")
20     return value

```

Рис. 4. Imports + OUT_DIR/OUT_FILE через `os.path.normpath` + функция `fmt(value)` для `Decimal`.

Нижняя часть скрипта — функция `main()` с блоком INSERT logs START → SELECT → `csv.writer` → INSERT logs END:

```

23 def main() -> None:
24     os.makedirs(OUT_DIR, exist_ok=True)
25     started = datetime.now()
26
27     with psycopg2.connect(**DB_CONFIG) as conn, conn.cursor() as cur:
28         cur.execute(
29             "INSERT INTO logs.etl_log(process,object_name,event,started_at,extra) "
30             "VALUES (%s,%s,%s,%s,%s)",
31             ("export_f101", "dm.dm_f101_round_f", "START", started, OUT_FILE),
32         )
33         cur.execute("SELECT * FROM dm.dm_f101_round_f ORDER BY ledger_account;")
34         cols = [c.name for c in cur.description]
35         rows = cur.fetchall()
36
37         with open(OUT_FILE, "w", encoding="utf-8", newline="") as f:
38             w = csv.writer(f, delimiter=";")
39             w.writerow(cols)
40             for r in rows:
41                 w.writerow([fmt(v) for v in r])
42
43         cur.execute(
44             "INSERT INTO logs.etl_log(process,object_name,event,started_at,finished_at,rows_affected,extra) "
45             "VALUES (%s,%s,%s,%s,%s,%s,%s)",
46             ("export_f101", "dm.dm_f101_round_f", "END",
47              started, datetime.now(), len(rows), OUT_FILE),
48         )
49
50     print(f"[OK] {OUT_FILE}: {len(rows)} строк")
51
52
53 if __name__ == "__main__":
54     main()
55

```

*Рис. 5. main(): connect, START-лог, SELECT * + cur.description, csv.writer(delimiter=";"), запись заголовка и строк через [fmt(v) for v in r], END-лог, print.*

4.1. Полный текст скрипта

"""Задача 1.4. Экспорт dm.dm_f101_round_f в CSV (с заголовком).

Формат: разделитель ';' (чтобы запятые в десятичных значениях, если вдруг появятся, не ломали парсинг), кодировка utf-8, первая строка — имена колонок таблицы. Decimal-значения форматируются как обычные числа с фиксированной точкой ('0.00000000'), без научной нотации ('0E-8'), чтобы CSV был читаемым в любом редакторе.

"""

```
from __future__ import annotations
import csv
import os
from datetime import datetime
from decimal import Decimal
import psycpg2
from db import DB_CONFIG

OUT_DIR = os.path.normpath(os.path.join(os.path.dirname(__file__), "..", "out"))
OUT_FILE = os.path.join(OUT_DIR, "dm_f101_round_f.csv")

def fmt(value):
    """Привести Decimal к обычной строке с фиксированной точкой."""
    if isinstance(value, Decimal):
        return format(value, "f")
    return value

def main() → None:
    os.makedirs(OUT_DIR, exist_ok=True)

    started = datetime.now()

    with psycpg2.connect(**DB_CONFIG) as conn, conn.cursor() as cur:
        cur.execute(
```

```

"INSERT INTO logs.etl_log(process,object_name,event,started_at,extra) "
"VALUES (%s,%s,%s,%s,%s)",
("export_f101", "dm.dm_f101_round_f", "START", started, OUT_FILE),
)
cur.execute("SELECT * FROM dm.dm_f101_round_f ORDER BY ledger_account;")
cols = [c.name for c in cur.description]
rows = cur.fetchall()
with open(OUT_FILE, "w", encoding="utf-8", newline="") as f:
w = csv.writer(f, delimiter=";")
w.writerow(cols)
for r in rows:
w.writerow([fmt(v) for v in r])
cur.execute(
"INSERT INTO logs.etl_log(process,object_name,event,started_at,finished_at,rows_affected,extra) "
"VALUES (%s,%s,%s,%s,%s,%s,%s)",
("export_f101", "dm.dm_f101_round_f", "END",
started, datetime.now(), len(rows), OUT_FILE),
)
print(f"[OK] {OUT_FILE}: {len(rows)} строк")
if __name__ == "__main__":
main()

```

5. Скрипт импорта (import_f101.py)

Логика скрипта:

- путь к источнику: либо `argv[1]`, либо `DEFAULT_FILE` (`../out/dm_f101_round_f.csv`); путь приводится к абсолютному через `os.path.abspath`;
- логируем событие START в `logs.etl_log` с `extra` = путь к импортируемому файлу;
- делаем `TRUNCATE TABLE dm.dm_f101_round_f_v2` — гарантирует идемпотентность импорта;
- открываем файл и через `cur.copy_expert(...)` запускаем серверный `COPY ... FROM STDIN`; опции: `FORMAT CSV`, `HEADER true` (первую строку трактуем как заголовок), `DELIMITER ';' и NULL` " (пустая строка → NULL, на случай если в CSV есть пустые ячейки);

- после COPY делаем SELECT count(*) — фактическое количество загруженных строк;
- логируем событие END с rows_affected = n;
- печатаем в stdout: «импортировано в dm.dm_f101_round_f_v2: N строк из <путь>».

```

1  from __future__ import annotations
2
3  import os
4  import sys
5  from datetime import datetime
6
7  import psycopg2
8
9  from db import DB_CONFIG
10
11  DEFAULT_FILE = os.path.normpath(
12      os.path.join(os.path.dirname(__file__), "..", "out", "dm_f101_round_f.csv")
13  )
14
15
16  def main() -> None:
17      src = os.path.abspath(sys.argv[1] if len(sys.argv) > 1 else DEFAULT_FILE)
18      started = datetime.now()
19
20      with psycopg2.connect(**DB_CONFIG) as conn, conn.cursor() as cur:
21          cur.execute(
22              "INSERT INTO logs.etl_log(process,object_name,event,started_at,extra) "
23              "VALUES (%s,%s,%s,%s,%s)",
24              ("import_f101", "dm.dm_f101_round_f_v2", "START", started, src),
25          )
26          cur.execute("TRUNCATE TABLE dm.dm_f101_round_f_v2;")
27          with open(src, "r", encoding="utf-8") as fh:
28              cur.copy_expert(
29                  "COPY dm.dm_f101_round_f_v2 FROM STDIN WITH (FORMAT CSV, HEADER true, DELIMITER ';', NULL '')",
30                  fh,
31              )
32          cur.execute("SELECT count(*) FROM dm.dm_f101_round_f_v2;")
33          n = cur.fetchone()[0]
34          cur.execute(
35              "INSERT INTO logs.etl_log(process,object_name,event,started_at,finished_at,rows_affected,extra) "
36              "VALUES (%s,%s,%s,%s,%s,%s,%s)",
37              ("import_f101", "dm.dm_f101_round_f_v2", "END",
38              started, datetime.now(), n, src),
39          )
40
41      print(f"[OK] импортировано в dm.dm_f101_round_f_v2: {n} строк из {src}")
42
43
44  if __name__ == "__main__":
45      main()
46

```

Рис. 6. `import_f101.py` — общий вид: `argv`-парсинг с `DEFAULT_FILE`, `START`-лог, `TRUNCATE`, `copy_expert("COPY ... FROM STDIN ... FORMAT CSV, HEADER true, DELIMITER ';', NULL '')`, `SELECT count(*)`, `END`-лог.

5.1. Полный текст скрипта

""""Задача 1.4. Импорт CSV обратно в dm.dm_f101_round_f_v2.

Импорт идёт через серверный COPY ... FROM STDIN: `psycopg2.cursor.copy_expert`

читает файл блоками и стримит их в Postgres — это самый быстрый путь

загрузки CSV. Перед COPY делаем TRUNCATE целевой таблицы — расчёт идемпотентен, скрипт можно запускать многократно. Для произвольного файла можно передать путь аргументом командной строки.

```
"""

from __future__ import annotations
import os
import sys
from datetime import datetime
import psycopg2
from db import DB_CONFIG
DEFAULT_FILE = os.path.normpath(
os.path.join(os.path.dirname(__file__), "..", "out", "dm_f101_round_f.csv")
)
def main() → None:
src = os.path.abspath(sys.argv[1] if len(sys.argv) > 1 else DEFAULT_FILE)
started = datetime.now()
with psycopg2.connect(**DB_CONFIG) as conn, conn.cursor() as cur:
cur.execute(
"INSERT INTO logs.etl_log(process,object_name,event,started_at,extra) "
"VALUES (%s,%s,%s,%s,%s)",
("import_f101", "dm.dm_f101_round_f_v2", "START", started, src),
)
cur.execute("TRUNCATE TABLE dm.dm_f101_round_f_v2;")
with open(src, "r", encoding="utf-8") as fh:
cur.copy_expert(
"COPY dm.dm_f101_round_f_v2 FROM STDIN WITH (FORMAT CSV, HEADER true, DELIMITER ';',
NULL) ",
fh,
)
```

```

cur.execute("SELECT count(*) FROM dm.dm_f101_round_f_v2;")

n = cur.fetchone()[0]

cur.execute(

"INSERT INTO logs.etl_log(process,object_name,event,started_at,finished_at,rows_affected,extra) "

"VALUES (%s,%s,%s,%s,%s,%s,%s)",

("import_f101", "dm.dm_f101_round_f_v2", "END",

started, datetime.now(), n, src),

)

print(f"[OK] импортировано в dm.dm_f101_round_f_v2: {n} строк из {src}")

if __name__ == "__main__":

    main()

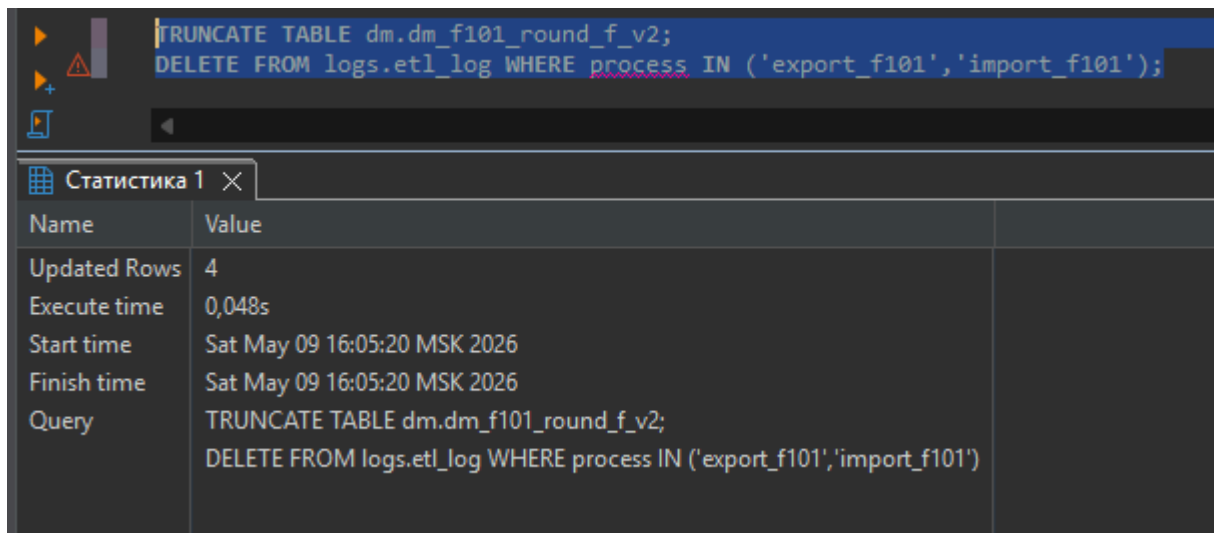
```

6. Чистый старт перед демонстрацией

Чтобы наглядно показать поведение обоих скриптов, перед запуском очищаем целевую таблицу _v2 и удаляем предыдущие лог-события export/import:

```
TRUNCATE TABLE dm.dm_f101_round_f_v2;
```

```
DELETE FROM logs.etl_log WHERE process IN ('export_f101','import_f101');
```



The screenshot shows the DBeaver SQL editor with the following SQL commands:

```

TRUNCATE TABLE dm.dm_f101_round_f_v2;
DELETE FROM logs.etl_log WHERE process IN ('export_f101','import_f101');

```

Below the editor, the 'Статистика 1' (Statistics 1) window is open, displaying the following data:

Name	Value
Updated Rows	4
Execute time	0,048s
Start time	Sat May 09 16:05:20 MSK 2026
Finish time	Sat May 09 16:05:20 MSK 2026
Query	TRUNCATE TABLE dm.dm_f101_round_f_v2; DELETE FROM logs.etl_log WHERE process IN ('export_f101','import_f101')

Рис. 7. TRUNCATE _v2 + DELETE логов в DBeaver. Updated Rows = 4 — это удалённые лог-записи от прошлых демо (TRUNCATE по своей природе строки в счётчик не пишет).

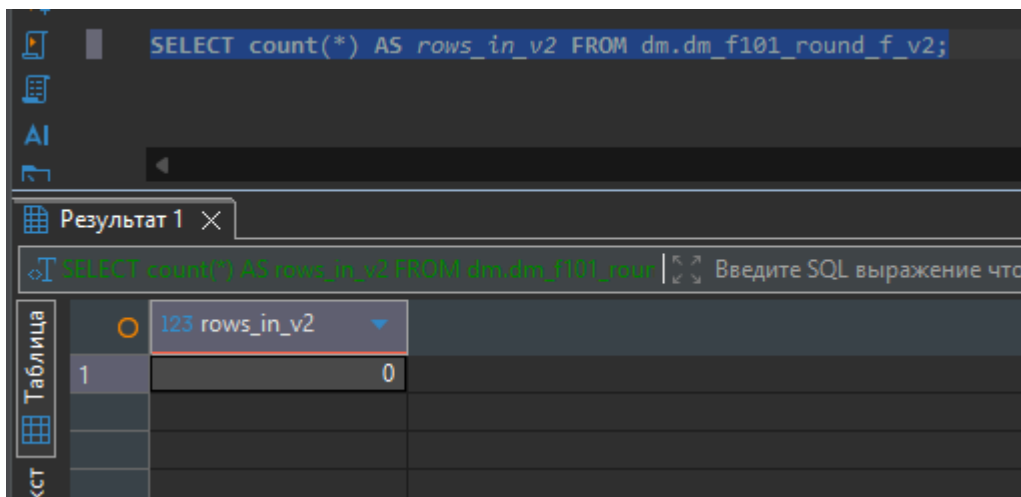


Рис. 8. Контрольный запрос — копия таблицы пуста: `rows_in_v2 = 0`.

7. Запуск экспорта

Запуск из терминала из директории `hw_coursework_task1/python`:

```
PS C:\Users\Maksim\Desktop\neo\hw_coursework_task1\python> python export_f101.py
[OK] C:\Users\Maksim\Desktop\neo\hw_coursework_task1\out\dm_f101_round_f.csv: 18 строк
PS C:\Users\Maksim\Desktop\neo\hw_coursework_task1\python> |
```

Рис. 9. `python export_f101.py` → [OK]
`C:\Users\Maksim\Desktop\neo\hw_coursework_task1\out\dm_f101_round_f.csv: 18 строк.`

Открываем получившийся CSV в Excel — видим 18 строк данных + заголовок, разделитель «;», все decimal-значения в обычном формате с 8 знаками после точки. Колонки идут ровно как в исходной таблице:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	from_date	to_date	chapter	ledger_aci	characteristic	balance_in	balance_in	balance_in	turn_deb	turn_deb	turn_deb	turn_deb	turn_cre	turn_cre	turn_cre	balance_out	balance_out	balance_out
2	01.01.2018	31.01.2018	❖❖	30102	❖❖	200079650	0.0000000	200079650	625440427	0.0000000	625440427	787848410	0.0000000	787848410	37671666	0.0000000	37671666	94000000
3	01.01.2018	31.01.2018	❖❖	30109	❖❖	63891.960	0.0000000	63891.960	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	63891.960	0.0000000	63891.960	00000000
4	01.01.2018	31.01.2018	❖❖	30110	❖❖	1347893.8	15170860	16518754	373332.34	0.0000000	373332.34	1083668.6	0.0000000	1083668.6	637557.52	15170860	15808418	41437800
5	01.01.2018	31.01.2018	❖❖	30111	❖❖	125006707	0.0000000	125006707	262884223	0.0000000	262884223	172294943	0.0000000	172294943	34417427	0.0000000	34417427	09000000
6	01.01.2018	31.01.2018	❖❖	30114	❖❖	0.0000000	146055279	146055279	0.0000000	146497265	146497265	0.0000000	0.0000000	219925136	219925136	0.0000000	726274074	726274074
7	01.01.2018	31.01.2018	❖❖	30126	❖❖	67224.060	0.0000000	67224.060	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	67224.060	0.0000000	67224.060	00000000
8	01.01.2018	31.01.2018	❖❖	30202	❖❖	129554.00	0.0000000	129554.00	0.0000000	0.0000000	0.0000000	0.0000000	93819.580	0.0000000	93819.580	35734.420	0.0000000	35734.420
9	01.01.2018	31.01.2018	❖❖	30204	❖❖	5541.8300	0.0000000	5541.8300	149647.04	0.0000000	149647.04	0.0000000	0.0000000	0.0000000	155188.87	0.0000000	155188.87	00000000
10	01.01.2018	31.01.2018	❖❖	30220	❖❖	0.0000000	142979349	142979349	0.0000000	251131748	251131748	0.0000000	162558540	162558540	0.0000000	134122026	134122026	62246384
11	01.01.2018	31.01.2018	❖❖	30221	❖❖	69885.070	21447412	21517297	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	69885.070	21447412	21517297	87254200
12	01.01.2018	31.01.2018	❖❖	30222	❖❖	215769.47	43368305	43584074	0.0000000	5595511.5	5595511.5	0.0000000	0.0000000	0.0000000	215769.47	37772793	37988563	28046400
13	01.01.2018	31.01.2018	❖❖	30223	❖❖	302098.75	0.0000000	302098.75	6349676.7	0.0000000	6349676.7	15781776	0.0000000	15781776	9734198.5	0.0000000	9734198.5	53000000
14	01.01.2018	31.01.2018	❖❖	30232	❖❖	86904.660	4517213.0	4604117.0	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	86904.660	4517213.0	4604117.0	72846400
15	01.01.2018	31.01.2018	❖❖	30233	❖❖	58966.570	5801448.0	5860414.6	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	58966.570	5801448.0	5860414.6	11544000
16	01.01.2018	31.01.2018	❖❖	30236	❖❖	39070.650	7152603.1	7191673.8	0.0000000	134845152	134845152	0.0000000	249074167	249074167	39070.650	121381618	121420688	79305900
17	01.01.2018	31.01.2018	❖❖	30413	❖❖	185530.94	0.0000000	185530.94	2098780.3	0.0000000	2098780.3	2382487.1	0.0000000	2382487.1	98175.85	0.0000000	98175.85	00000000
18	01.01.2018	31.01.2018	❖❖	30424	❖❖	7855306.5	0.0000000	7855306.5	4414156.6	0.0000000	4414156.6	9299232.4	0.0000000	9299232.4	2970230.7	0.0000000	2970230.7	71000000
19	01.01.2018	31.01.2018	❖❖	30425	❖❖	87620.470	2207131.9	2294752.4	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	87620.470	2207131.9	2294752.4	16262600

Рис. 10. `dm_f101_round_f.csv` в Excel — 18 строк, первая строка-заголовок (`from_date`, `to_date`, `chapter`, `ledger_account`, `characteristic`, `balance_in` *, `turn_deb` *, `turn_cre` *, `balance_out` *).

Замечание про отображение глав/характеристик «А»/«П» в Excel: по умолчанию Excel открывает CSV в кодировке `sr1251` и кириллический символ «А» отображается как «?/?» — это особенность открытия в Excel, а не проблема CSV. Сам файл сохранён в `utf-8`, что и читается напрямую через `Postgres COPY` (см. далее). Чтобы увидеть кириллицу в Excel, нужно использовать импорт через `Data → From Text/CSV` с указанием кодировки `UTF-8`.

8. Подмена двух значений в копии CSV

Чтобы проверить, что импорт реально подхватывает изменения из файла, делаем копию выгрузки `dm_f101_round_f_modified.csv` и меняем в ней пару значений:

- строка с `ledger_account = 30102`: `balance_out_total` с 37 671 666.94 на 99 999 999.99 (и `balance_out_rub` аналогично, потому что в исходных данных `val=0` и `total=rub`);
- строка с `ledger_account = 30220`: `balance_in_val` (он же `balance_in_total`, поскольку этот счёт валютный) с 1 429 793 492.98 на 1 111 111 111.11.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	from_date	to_date	chapter	ledger_aci	character	balance_i	balance_j	balance_i	turn_deb	turn_deb	turn_deb	turn_cre	turn_cre	turn_cre	balance_c	balance_c	balance_out_total	
2	01.01.2018	31.01.2018	♦♦	30102	♦♦	200079650	0.0000000	200079650	625440427	0.0000000	625440427	787848410	0.0000000	787848410	999999999	0.0000000	999999999	990000000
3	01.01.2018	31.01.2018	♦♦	30109	♦♦	63891.960	0.0000000	63891.960	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	63891.960	0.0000000	63891.96000000	
4	01.01.2018	31.01.2018	♦♦	30110	♦♦	1347893.8	15170860.	16518754.	373332.34	0.0000000	373332.34	1083668.6	0.0000000	1083668.6	637557.52	15170860.	15808418.4	41437800
5	01.01.2018	31.01.2018	♦♦	30111	♦♦	125006707	0.0000000	125006707	262884223	0.0000000	262884223	172294943	0.0000000	172294943	34417427.	0.0000000	34417427.	090000000
6	01.01.2018	31.01.2018	♦♦	30114	♦♦	0.0000000	146055275	146055275	0.0000000	146497265	146497265	0.0000000	219925136	219925136	0.0000000	726274074	726274074.	37950655
7	01.01.2018	31.01.2018	♦♦	30126	♦♦	67224.060	0.0000000	67224.060	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	67224.060	0.0000000	67224.06000000	
8	01.01.2018	31.01.2018	♦♦	30202	♦♦	129554.00	0.0000000	129554.00	0.0000000	0.0000000	0.0000000	93819.580	0.0000000	93819.580	35734.420	0.0000000	35734.42000000	
9	01.01.2018	31.01.2018	♦♦	30204	♦♦	5541.8300	0.0000000	5541.8300	149647.04	0.0000000	149647.04	0.0000000	0.0000000	0.0000000	155188.87	0.0000000	155188.87000000	
10	01.01.2018	31.01.2018	♦♦	30220	♦♦	0.0000000	111111111	111111111	0.0000000	251131748	251131748	0.0000000	162558540	162558540	0.0000000	134122028	134122028.	62246384
11	01.01.2018	31.01.2018	♦♦	30221	♦♦	69885.070	21447412.	21517297.	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	69885.070	21447412.	21517297.	87254200
12	01.01.2018	31.01.2018	♦♦	30222	♦♦	215769.47	43368305.	43584074.	0.0000000	5595511.5	5595511.5	0.0000000	0.0000000	0.0000000	215769.47	37772793.	37988563.	28046400
13	01.01.2018	31.01.2018	♦♦	30223	♦♦	302098.75	0.0000000	302098.75	6349676.7	0.0000000	6349676.7	15781776.	0.0000000	15781776.	9734198.5	0.0000000	9734198.5	30000000
14	01.01.2018	31.01.2018	♦♦	30232	♦♦	86904.660	4517213.0	4604117.7	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	86904.660	4517213.0	4604117.7	2846400
15	01.01.2018	31.01.2018	♦♦	30233	♦♦	58966.570	5801448.0	5860414.6	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	58966.570	5801448.0	5860414.6	1154400
16	01.01.2018	31.01.2018	♦♦	30236	♦♦	39070.650	7152603.1	7191673.8	0.0000000	134845152	134845152	0.0000000	249074167	249074167	39070.650	121381618	121420688.	79305900
17	01.01.2018	31.01.2018	♦♦	30413	♦♦	185530.94	0.0000000	185530.94	2098780.3	0.0000000	2098780.3	2382487.1	0.0000000	2382487.1	-98175.85	0.0000000	-98175.85	00000000
18	01.01.2018	31.01.2018	♦♦	30424	♦♦	7855306.5	0.0000000	7855306.5	4414156.6	0.0000000	4414156.6	9299232.4	0.0000000	9299232.4	2970230.7	0.0000000	2970230.7	10000000
19	01.01.2018	31.01.2018	♦♦	30425	♦♦	87620.470	2207131.9	2294752.4	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	87620.470	2207131.9	2294752.4	2162600

Рис. 11. `dm_f101_round_f_modified.csv` в Excel — видны две подмены: для 30102 в столбце `balance_out_total` стало 99 999 999.99, для 30220 в столбце `balance_in_val/total` — 1 111 111 111.11.

9. Запуск импорта

Запускаем `import_f101.py` с явным указанием пути к модифицированному файлу:

```
PS C:\Users\Maksim\Desktop\neo\hw_coursework_task1\python> python import_f101.py ../out/dm_f101_round_f_modified.csv
[OK] импортировано в dm.dm_f101_round_f_v2: 18 строк из C:\Users\Maksim\Desktop\neo\hw_coursework_task1\out\dm_f101_round_f_modified.csv
```

Рис. 12. `python import_f101.py ../out/dm_f101_round_f_modified.csv → [OK]` импортировано в `dm.dm_f101_round_f_v2`: 18 строк из `C:\Users\Maksim\Desktop\neo\hw_coursework_task1\out\dm_f101_round_f_modified.csv`.

Контрольный запрос подтверждает, что в `_v2` теперь те же 18 строк:

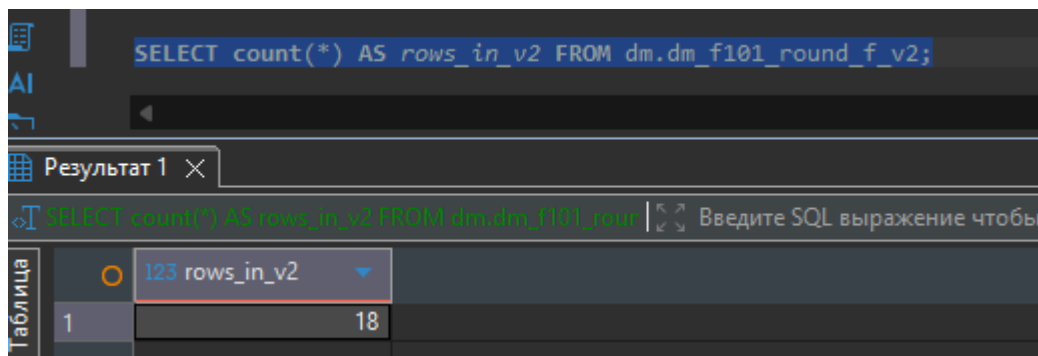
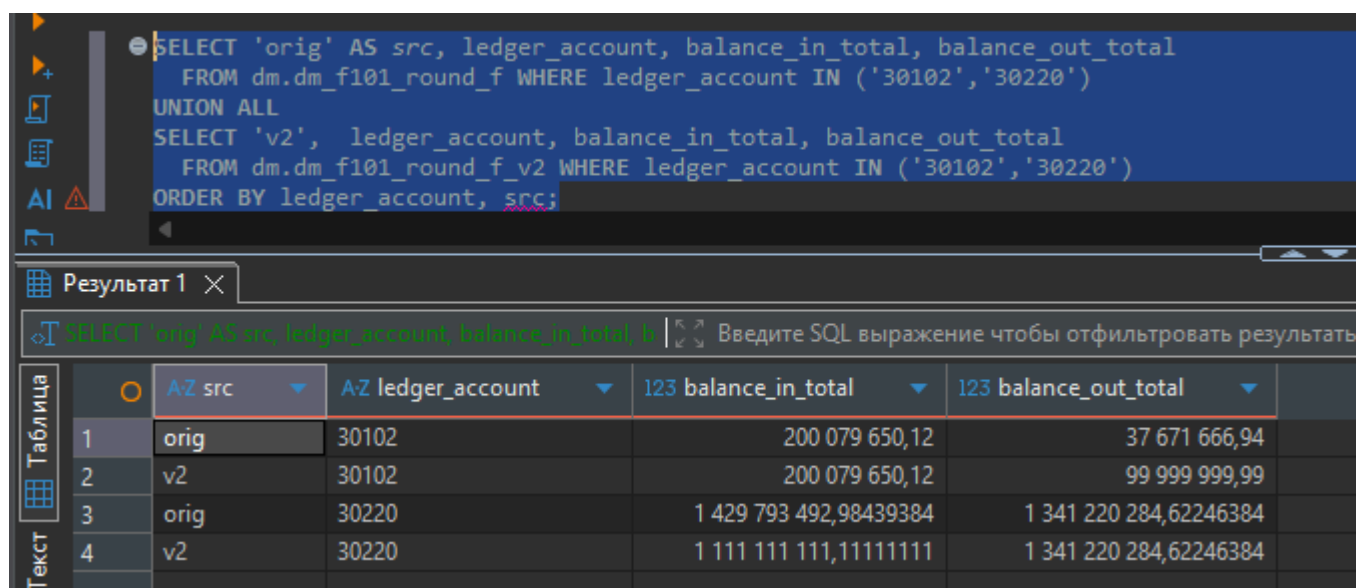


Рис. 13. `SELECT count(*) FROM dm.dm_f101_round_f_v2 → rows_in_v2 = 18`.

10. Сверка: исходная витрина vs копия после импорта

Сравниваем содержимое dm.dm_f101_round_f (оригинал) и dm.dm_f101_round_f_v2 (после импорта изменённого CSV) для двух подменённых строк. В выдаче должны быть видны ровно те значения, которые мы вписали в CSV:

```
SELECT 'orig' AS src, ledger_account, balance_in_total, balance_out_total
FROM dm.dm_f101_round_f WHERE ledger_account IN ('30102','30220')
UNION ALL
SELECT 'v2', ledger_account, balance_in_total, balance_out_total
FROM dm.dm_f101_round_f_v2 WHERE ledger_account IN ('30102','30220')
ORDER BY ledger_account, src;
```



src	ledger_account	balance_in_total	balance_out_total
orig	30102	200 079 650,12	37 671 666,94
v2	30102	200 079 650,12	99 999 999,99
orig	30220	1 429 793 492,98439384	1 341 220 284,62246384
v2	30220	1 111 111 111,11111111	1 341 220 284,62246384

Рис. 14. Сравнение «orig vs v2» по двум счетам. 30102: balance_out_total 37 671 666.94 → 99 999 999.99 (наша подмена). 30220: balance_in_total 1 429 793 492.98 → 1 111 111 111.11 (наша подмена). Третье поле (balance_out_total для 30220) не менялось — значения совпадают, что подтверждает, что импорт не «затёр» остальные данные.

11. Логирование

Оба скрипта пишут события в общую таблицу logs.etl_log. На каждый запуск — пара START/END: process = export_f101 или import_f101, object_name = соответствующая таблица в БД, в extra — путь к CSV-файлу, в rows_affected у END — число обработанных строк.

```

SELECT log_id, process, event,
       round(EXTRACT(EPOCH FROM finished_at - started_at)::numeric, 3) AS dur_sec,
       rows_affected, extra
FROM logs.etl_log
WHERE process IN ('export_f101', 'import_f101')
ORDER BY log_id;

```

	log_id	process	event	dur_sec	rows_affected	extra
1	421	export_f101	START	[NULL]	[NULL]	C:\Users\Maksim\Desktop\neo\hw_coursework_task1\out\dm_f101_round_f.csv
2	422	export_f101	END	0,054	18	C:\Users\Maksim\Desktop\neo\hw_coursework_task1\out\dm_f101_round_f.csv
3	423	import_f101	START	[NULL]	[NULL]	C:\Users\Maksim\Desktop\neo\hw_coursework_task1\out\dm_f101_round_f_modified.csv
4	424	import_f101	END	0,054	18	C:\Users\Maksim\Desktop\neo\hw_coursework_task1\out\dm_f101_round_f_modified.csv

Рис. 15. logs.etl_log по process IN (export_f101, import_f101): четыре записи — пара (START 421 + END 422) для экспорта и пара (START 423 + END 424) для импорта; rows_affected=18, dur_sec≈0.054 с у END-записей; в extra — полные пути к CSV-файлам.

12. Итог по задаче 1.4

- написан скрипт export_f101.py: SELECT * FROM dm.dm_f101_round_f → CSV (delimiter=";", utf-8, первая строка — имена колонок); Decimal-значения форматируются как «0.00000000», без научной нотации;
- написан скрипт import_f101.py: TRUNCATE _v2 + серверный COPY ... FROM STDIN с опциями FORMAT CSV, HEADER true, DELIMITER ';', NULL ""; путь к файлу — необязательный аргумент командной строки;
- демо подмены: после изменения двух значений в копии CSV (30102.balance_out_total и 30220.balance_in_total) и запуска импорта в dm.dm_f101_round_f_v2 эти значения отличаются от оригинальной витрины — значит импорт корректно подхватывает изменения файла;
- идемпотентность импорта: TRUNCATE перед COPY гарантирует, что повторный запуск не создаст дубликаты;
- логирование START/END ведётся в единую таблицу logs.etl_log (process = export_f101 / import_f101); фактическая длительность считается арифметикой timestamp'ов finished_at – started_at.

Этим закрываются все требования задачи 1.4 курсовой работы.

13. Ссылки

Репозиторий GitHub (ветка 1_4): https://github.com/kodi842/neoflex_coursework/tree/1_4

Видео-демонстрация (Google Drive, все 4 видео по курсовой): <https://drive.google.com/drive/folders/1ZMhXnrIE3nR8zUqTHC2uECeeShwDaIFG?usp=sharing>